

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I Athavan K (192524036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student : Athavan k

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain

subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1 — AI-Powered Drones for Flood-Relief Medicine Delivery

Scenario: A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic — roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has a heuristic estimate (straight-line distance), partial real-time updates about blocked paths, and variable movement costs depending on terrain and weather.

i. Behaviour of Greedy Best-First Search (GBFS)

GBFS selects the next node to expand purely on the basis of the heuristic function $h(n)$ — the straight-line distance to the destination — while completely ignoring the cost already incurred, $g(n)$. In this scenario, the drone would always move toward the neighbouring waypoint that appears geometrically closest to the flood-relief target.

- Strengths: Very fast and lightweight, since it expands the minimum number of nodes needed to reach a goal-looking direction; this low latency suits the urgency of an emergency delivery where a decision is needed almost instantly.
- Strengths: Requires little memory/computation, which matters for a resource-constrained embedded drone controller.
- Weaknesses: It is neither complete nor optimal — because it disregards accumulated cost, it can commit to a route that looks short on a straight line but is actually blocked, flooded, or traverses high-risk terrain.
- Weaknesses: Under uncertainty (partial real-time updates), GBFS has no mechanism to weigh the real risk/cost of a path; it can be greedily lured toward a dead end and then have to replan late, wasting critical time during an emergency.

ii. Behaviour of A* Search

A* evaluates every node using $f(n) = g(n) + h(n)$, combining the actual cost accumulated so far (which can reflect real terrain difficulty and weather-adjusted travel cost) with the heuristic estimate of the remaining distance. This lets the drone balance 'how much it has already spent' against 'how much farther it likely has to go.'

- Because the straight-line distance heuristic never overestimates the true road distance, it is admissible, so A* is guaranteed to find the optimal (least-cost) route whenever one exists and is known.
- A* naturally reflects variable movement cost: a longer path over safe, clear terrain can be preferred over a shorter path through a high-cost flooded/stormy area, because $g(n)$ captures real terrain and weather cost, not just distance.
- When combined with an incremental replanning technique (e.g., D* Lite / Lifelong Planning A*), A* can repair only the affected part of its search tree when a road becomes blocked, instead of restarting from scratch.

iii. Impact of Heuristic Accuracy on Both Algorithms

Heuristic accuracy affects GBFS and A* very differently because of how each algorithm uses it.

- In GBFS, the heuristic is the sole driver of the search. An inaccurate or misleading heuristic (e.g., a straight line that ignores a flooded river crossing) can send the drone directly into danger or into long detours, since there is no real-cost term to correct the mistake.
- In A*, the heuristic only guides the search — $g(n)$ keeps the algorithm grounded in real, already-incurred cost. A weaker heuristic degrades A*'s efficiency (it approaches the behaviour of Uniform-Cost Search, expanding more nodes) but does not break its optimality as long as the heuristic remains admissible and consistent.
- If the heuristic were to overestimate true cost (inadmissible), A*'s optimality guarantee would be lost — a serious concern for a life-critical application where the 'optimal' route also implies the safest and fastest one.

iv. Effect of Dynamic Changes (Blocked Paths) on Search Performance

- Both algorithms are designed around a static-graph assumption; a sudden blockage invalidates part of the previously computed path and forces replanning.
- Plain re-running of either algorithm from scratch after every update is wasteful; incremental/real-time search variants (D* Lite, LRTA*) instead repair only the affected region of the search graph, which is essential to keep response time low.
- GBFS is hit harder by dynamic blockages: because it commits greedily to the heuristically closest option, a block often forces it into large detours and repeated backtracking with no memory of accumulated cost to fall back on.
- A* degrades more gracefully because $g(n)$ allows it to compare the blocked route's already-incurred cost against alternative routes systematically, producing a more reasoned detour.

v. Recommendation and Justification

A* (preferably an incremental, real-time variant such as D* Lite, or a bounded/weighted A* when strict time limits are critical) is the most suitable algorithm for this scenario.

- **Optimality & Safety:** A* accounts for actual terrain/weather cost, not just geometric distance, so it avoids routes that are short on paper but hazardous or slow in reality — vital when the cargo is emergency medicine.
- **Real-time constraints:** A weighted/bounded variant of A* can trade a small, controlled amount of optimality for guaranteed faster decisions, keeping response time within the drone's operational limits.
- **Adaptability:** Its incremental replanning variants efficiently absorb partial real-time updates about blocked paths without recomputing the entire route, which GBFS cannot do in a cost-aware manner.

Overall, A* (or its incremental/weighted variants) offers the best balance of speed, safety, and route optimality, whereas GBFS should only be considered as a fallback for extremely time-critical decisions where a rough estimate is acceptable.

Question 2 — Smart City Traffic Signal Optimization

Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections, minimizing total waiting time, fuel consumption, and congestion, while continuously adjusting to dynamic traffic flow in an extremely large search space that risks getting stuck in local optima.

Formulation as an Optimization Problem

- **State:** a complete assignment of signal-timing parameters (green/red durations, phase offsets) across all intersections at a given moment.
- **Objective function:** a weighted combination of total waiting time, fuel consumption, and congestion level, to be minimized (equivalently, a 'fitness' to be maximized).
- **Search space:** combinatorially huge — every intersection contributes several continuous/discrete timing variables, and the number of feasible combinations grows exponentially with the number of intersections.

Traditional systematic search (BFS, DFS, uniform-cost search) is inefficient here because there is no single well-defined 'goal state' to search toward — the task is continuous improvement of a configuration, not finding a path in a graph. The search space is too large to enumerate, traffic patterns change dynamically (making any pre-computed exhaustive solution stale almost immediately), and the true objective landscape is non-convex, containing many locally optimal configurations. This calls for local/metaheuristic search methods that iteratively refine a current candidate solution rather than explicitly exploring a path-based state space.

i. Hill Climbing in This Scenario

Hill Climbing starts from a candidate signal-timing configuration and repeatedly moves to a neighbouring configuration (e.g., a small change in one intersection's green-light duration) if and only if it improves the objective function, stopping when no neighbour is better.

- Its main limitation is myopia: it only considers immediate neighbours and has no mechanism to accept a temporarily worse configuration, so it easily settles into a local optimum, a flat plateau, or a ridge, rather than the global optimum across hundreds of interacting intersections.
- It also has no memory or restart mechanism by default, so results depend heavily on the starting configuration.

ii. Local Maxima, Plateaus, and Ridges — Real-World Interpretation

- Local maxima: a timing plan that looks optimal in isolation for one intersection can be far from city-wide optimal, because further improvement requires simultaneously adjusting several neighbouring intersections together — a move Hill Climbing, limited to single-step changes, cannot make, so it gets 'trapped' believing it has found the best plan.
- Plateaus: during low-traffic periods (e.g., late night), many different timing configurations yield nearly identical waiting times, so the algorithm wanders across equally-scored configurations without any signal of which direction is genuinely better.
- Ridges: improving flow in one direction (e.g., north-south) often requires a coordinated change in another direction (e.g., east-west) at the same time; because Hill Climbing changes only one variable/intersection at a time, it can zig-zag along the ridge, converging very slowly or stalling.

iii. Overcoming These Limitations with Simulated Annealing / Genetic Algorithms

- Simulated Annealing (SA) allows the search to occasionally accept a worse configuration, with a probability that decreases as a 'temperature' parameter cools over time. Early on it explores broadly (escaping local maxima and plateaus), and later it exploits the best region found, converging toward a near-global optimum.
- Genetic Algorithms (GA) maintain a population of candidate timing plans and apply selection, crossover, and mutation across generations. Crossover combines good partial solutions from different intersections/regions of the city, while mutation injects diversity that helps escape local maxima and ridges; the population-based approach naturally explores multiple regions of the huge search space in parallel.
- Both techniques add controlled randomness that Hill Climbing lacks, directly targeting its core weakness — the inability to move past locally optimal but globally inferior configurations.

iv. Recommended Approach and Justification

A hybrid strategy is recommended: Genetic Algorithms for periodic, city-wide (offline/batch) optimization of base signal-timing plans, combined with Simulated Annealing (or fast Hill Climbing seeded from the GA solution) for rapid, localized real-time adjustments in response to sudden traffic surges or incidents.

- Scalability: GA's population-based, parallelizable search handles the combinatorial scale of hundreds of intersections far better than a single-trajectory method, and multi-objective GA variants can directly optimize waiting time, fuel, and congestion together.
- Adaptability: SA (or a warm-started Hill Climbing) is lightweight enough to re-tune a subset of intersections quickly as live traffic data arrives, without re-running a full generational GA cycle.

This combination balances solution quality (global, well-optimized base plans from GA) with responsiveness (fast local correction via SA), which best matches the smart city's requirements for scalability and adaptability.

Question 3 — Autonomous Mars Rover Exploration

Scenario: An autonomous rover explores unknown Martian terrain, navigating to collect samples, avoiding hazards such as craters and rocks, operating with limited sensing, and making decisions without a complete map — updating its knowledge as it explores.

i. Why an Online Search Agent (Not Offline Search) Is Required

Offline search assumes the agent has complete, accurate knowledge of the environment before it starts acting, and it computes an entire solution path in advance, executing it blindly. On Mars, the terrain map is incomplete, sensing range is limited, and hazards are only discovered as the rover physically explores — so no complete model exists beforehand, making pure offline search infeasible.

An online search agent instead interleaves computation with action: it senses, acts, observes the outcome, updates its internal knowledge, and re-plans — the only viable approach when the environment must be discovered through acting in it.

ii. Challenges of Partial Observability and Dynamic Environments

- Partial observability: the rover can only sense a limited local region around itself, so it may commit to a direction that later reveals a hazard (a crater or rock) just beyond its current sensing range, forcing costly re-routing or, in the worst case, irreversible damage.
- Communication delay: light-time delay to Earth means the rover frequently must decide autonomously without waiting for human confirmation, increasing the importance of safe, self-contained decision-making.
- Exploration vs. exploitation trade-off: the rover must balance discovering new terrain (to build a better model) against making direct progress toward its sample-collection goals.
- Irreversibility: unlike a simulated search tree, some real-world actions (e.g., driving into a crater) cannot be 'backtracked,' so risk-aware decision-making is essential, not just cost-minimization.

iii. How the Rover Builds and Updates Its Internal Model

- The rover maintains an internal partial map (e.g., an occupancy grid or graph of visited/observed states), updated after every sensing and movement action.
- Algorithms such as Learning Real-Time A* (LRTA*) update the heuristic estimate associated with each visited state based on the actual cost experienced getting there, so the rover's notion of 'distance/cost to goal' becomes progressively more accurate the more it explores.
- Newly sensed obstacles are merged into the map, previously assumed-passable edges are marked blocked, and the rover's next decision is made using this continuously refined model rather than a fixed, pre-computed one.

iv. Online Search vs. Uninformed and Informed Offline Search

- Uninformed search (e.g., BFS/DFS/uniform-cost) explores blindly with no heuristic guidance and, more critically, assumes the full state space is known — unusable for an environment the rover has not yet observed, and computationally very expensive at real terrain scale.
- Informed offline search (e.g., A*) is more efficient thanks to heuristic guidance, but it still requires a complete map computed in advance; it cannot function when large parts of the map are unknown until discovered.
- Online informed search (e.g., LRTA*) merges the efficiency of heuristic guidance with the ability to act under incomplete knowledge, continually revising both its plan and its heuristic estimates as new information arrives — the only category of the three that is actually workable for Mars exploration.

v. Justification of the Most Suitable Approach

An online, informed search approach such as LRTA* (extended with a safety- or risk-aware cost function that penalizes proximity to hazards) is the most suitable choice.

- Uncertainty: it operates correctly despite an initially unknown map, updating its knowledge only as the rover senses its surroundings.
- Adaptability: it continuously revises both its internal model and its plan, so newly discovered obstacles are incorporated immediately rather than invalidating a fixed pre-computed route.
- Safety: hazard-avoidance can be embedded directly into the cost function (e.g., high cost near craters/rocks), biasing the rover away from risky, unexplored terrain even under partial observability.

This makes LRTA*-style online search clearly superior to offline uninformed or informed search for this exploration task.

Question 4 — AI-Generated University Exam Timetable

Scenario: A university must automatically generate exam timetables that satisfy no overlapping exams for any student, limited exam-hall availability, prerequisite scheduling order between certain subjects, faculty availability constraints, and special accommodations for some students, with complexity growing as courses and students increase.

Formulation as a Constraint Satisfaction Problem (CSP)

i. Variables, Domains, and Constraints

- Variables: one variable per exam, e.g., $X_1, X_2, \dots, X_n$, each representing the exam of a specific course/subject that needs to be scheduled.
- Domains: for each variable, the domain is the set of feasible (time-slot, exam-hall) pairs available during the examination period, e.g., {Day1-9AM-HallA, Day1-9AM-HallB, Day1-2PM-HallA, Day2-9AM-HallA, ...}.
- Unary constraints: some exams may have a fixed slot mandated in advance (e.g., accreditation or external-body requirements), restricting a single variable's domain directly.
- Binary constraints (no overlaps): for any two courses that share at least one common student, their assigned time slots must be different — this is the classic 'no student has two exams at once' rule.
- Resource (hall-capacity) constraints: the number of exams scheduled in the same slot cannot exceed the number of available exam halls.
- Precedence constraints: certain subjects must be scheduled strictly before others (ordering constraints between pairs of exam variables).
- Faculty-availability constraints: the invigilating faculty assigned to an exam must be free during that exam's slot, treating faculty as another shared, constrained resource similar to halls.
- Accommodation constraints: students requiring extended time or a separate, quieter hall impose additional constraints on the slot/hall chosen for their specific exams.

ii. How Backtracking Search Works in This Scenario

Backtracking search assigns a (time-slot, hall) value to one exam variable at a time, following a variable-ordering heuristic (e.g., Minimum Remaining Values — choosing the exam with the fewest legal slots left — or the Degree Heuristic — choosing the exam involved in the most constraints with other exams, since university timetables have many highly-connected 'popular elective' courses).

After every assignment, the algorithm checks whether all relevant constraints (no student overlap, hall capacity, precedence, faculty availability) remain satisfiable. If a violation is detected, the search backtracks to the most recently assigned variable and tries the next value in its domain, continuing until either a complete, consistent timetable is produced or the search space is exhausted.

iii. How Constraint Propagation (Forward Checking) Improves Efficiency

- Forward checking, applied after each assignment, removes now-inconsistent values from the domains of yet-unassigned variables that are directly connected to the just-assigned exam (e.g., once Exam A is fixed to Day1-9AM-HallX, that slot/hall combination — and any conflicting slot for exams sharing students with A — is removed from related exams' domains).
- This detects dead-ends much earlier than plain backtracking, which would otherwise only discover the same conflict many assignments later, deep in the search tree, and waste significant time undoing work.
- Stronger propagation, such as arc-consistency (AC-3), goes further by ensuring every remaining value in a variable's domain has at least one compatible value in every connected variable's domain, pruning even more of the search space before backtracking is ever needed.
- Overall, propagation techniques turn an otherwise exponential search into a far more tractable one, which is essential as the number of courses and students — and hence constraints — grows.

iv. Real-World Challenges and Best Strategy for Scalability

- Challenges: at university scale, the constraint graph becomes extremely dense (a single popular elective can connect to nearly every other exam through shared students); multiple simultaneous resource constraints (halls, faculty, accommodations) increase the effective branching factor; and late changes (e.g., a faculty member becoming unavailable) may require re-scheduling parts of an already-solved timetable without recomputing everything from scratch.
- Recommended strategy: combine backtracking with strong heuristics — MRV and the Degree Heuristic for variable ordering, Least-Constraining-Value for value ordering — together with constraint propagation (forward checking or AC-3) to prune early. For very large instances, decompose the problem (e.g., by department, year, or semester) into weakly-connected sub-timetables to shrink the effective constraint graph, and use local-search repair methods such as Min-Conflicts to handle late-breaking changes quickly by adjusting only the affected exams instead of re-solving the entire CSP.

This combination of heuristic-guided backtracking, propagation, decomposition, and local repair gives the best balance of solution quality and scalability as the university's course and student population grows.

Question 5 — Strategic Game AI: Minimax and Alpha-Beta Pruning

Scenario: A gaming company is developing an AI opponent for a real-time strategy game that must compete against human players, make decisions within strict time limits, and evaluate complex game states with an extremely large decision tree, making full computation expensive.

i. How the Minimax Algorithm Models This Problem

Minimax models the game as a tree of alternating turns: the AI is the MAX player, trying to choose moves that maximize its evaluation score, while the opponent is treated as the MIN player, assumed to play optimally to minimize that same score. The algorithm recursively explores possible move sequences down to a cutoff depth (or terminal state), then propagates evaluation values back up the tree — MAX nodes take the maximum value among their children, MIN nodes take the minimum — so the root's chosen move is the one that guarantees the best outcome for the AI even against a worst-case (optimal) opponent.

ii. Computational Challenges of Large Game Trees

- The real-time strategy game's decision tree has both a large branching factor (many possible unit/action combinations per turn) and large depth, giving a time complexity of $O(b^d)$, which becomes computationally intractable to search exhaustively.
- Strict time limits mean the AI cannot search to terminal game states; it must cut the search off at a limited depth and rely on a heuristic evaluation function, trading some decision accuracy for speed.

- Memory can also become a bottleneck, particularly if storing large trees, move histories, or transposition tables to detect repeated states.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta pruning enhances Minimax by tracking two bounds while traversing the tree: alpha, the best score MAX can currently guarantee, and beta, the best score MIN can currently guarantee. A branch is pruned — skipped entirely — as soon as it is proven that it cannot affect the final decision at the root.

- Detailed reasoning: while exploring a MIN node's children, if a value $\leq$ alpha is found, the rest of that branch can be skipped, because MAX already has a better option elsewhere and would never let the game reach this MIN node in the first place; the symmetric argument (value $\geq$ beta) applies at MAX nodes with respect to MIN's existing guarantee.
- Alpha-Beta pruning computes exactly the same result as plain Minimax — it never changes the chosen move — it simply avoids expanding branches that provably cannot change that result.
- With good move ordering (trying likely-best moves first), Alpha-Beta reduces the effective branching factor from b to roughly $\sqrt{b}$, which in practice allows the AI to search roughly twice as deep within the same time budget, directly addressing the strict real-time constraint.

iv. Designing an Evaluation Function

Because the search is cut off before terminal states, a heuristic evaluation function is needed to estimate how favourable a given, non-terminal game state is for the AI. For a real-time strategy game, relevant factors include:

- Material/unit balance — the number and strength of units controlled by each side.
- Economic strength — resource stockpiles and production capacity, since RTS games are won as much by economy as by combat.
- Territorial control — map area and strategic positions (e.g., resource nodes, choke points) held by each side.
- Positional advantage — unit positioning relative to terrain features such as high ground or narrow passages.
- Tempo/mobility — the potential for future actions, such as production capacity and unit mobility, which influence how quickly a side can respond to threats.
- Threat level — imminent attacks on, or vulnerabilities of, each side's key assets.

Each factor is assigned a weight reflecting its strategic importance, and these are combined into a single numeric score, ideally tuned empirically (e.g., through playtesting or machine learning against recorded games) rather than fixed arbitrarily. A well-designed evaluation function is what makes limited-depth search effective in practice, since it approximates the 'true' long-term value of a state without requiring the search to reach the actual end of the game.

v. Strategy for Balancing Optimality and Real-Time Performance

- Use Alpha-Beta pruning combined with iterative deepening — searching progressively deeper layer by layer — so the AI always has a valid best move available even if the time budget is exhausted mid-search.
- Apply move-ordering heuristics (e.g., trying previously successful moves first, killer-move heuristics) to maximize the pruning achieved by Alpha-Beta.
- Add quiescence search at volatile/unstable positions (e.g., mid-battle) to avoid stopping evaluation at a misleadingly favourable or unfavourable moment.
- Invest in a well-tuned evaluation function, since search depth alone cannot compensate for a poor heuristic under a tight time budget.

Together, these techniques allow the AI to consistently produce near-optimal decisions within the game's strict real-time constraints, balancing computational cost against strategic strength.